

2.5 自定义接口 (Interfaces)

所属阶段： 第二阶段 · ROS 2 核心通信机制

讲次数量： 6 节

预计时长： ~35 分钟

前置依赖： 2.4 动作 (Action) 通信

学习目标

完成本节后，学员将能够：

- 解释 ROS 2 三种接口类型 (`.msg` 、 `.srv` 、 `.action`) 的作用与命名规则
- 创建纯接口功能包 `hardware_interfaces` ，编写并构建自定义 `Msg` 定义文件
- 在 Python 节点中导入并使用 `HardwareStatus` 消息类型进行发布与订阅
- 在 C++ 节点中导入并使用 `HardwareStatus` 消息类型进行发布与订阅
- 在接口包中添加自定义 `Srv` 定义 `ComputeRectangleArea.srv` 并完成构建
- 运用 `ros2 interface show` 、 `ros2 interface list` 、 `ros2 interface package` 命令查看与调试接口

命名说明：本节使用 `hardware_interfaces` 作为独立练习包；2.7 项目会重新创建 `my_robot_interfaces` 作为项目专用接口包，避免两个章节的接口定义互相覆盖。

讲次 1：什么是 ROS 2 Interfaces

学习目标

- 理解 Interface（接口）在 ROS 2 通信体系中的核心地位
- 区分 `.msg`、`.srv`、`.action` 三种接口文件及其对应的通信机制
- 掌握接口类型的完整命名格式与代码导入方式

知识点

为什么需要自定义接口

在 2.2-2.4 中，我们使用了大量**标准接口**：

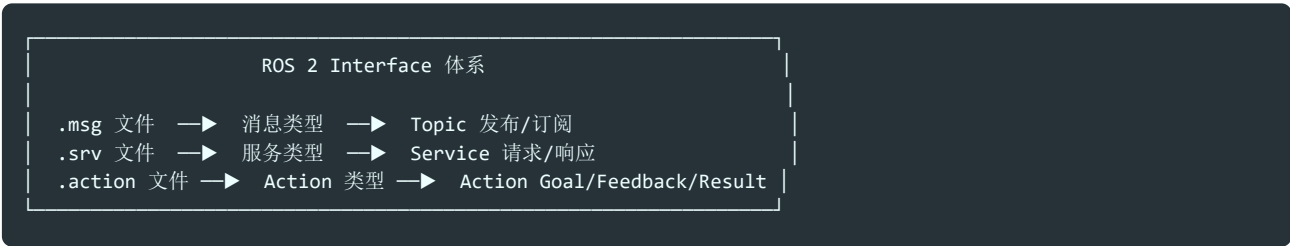
通信机制	标准接口示例	用途
Topic	<code>std_msgs/msg/String</code>	发布文本
Topic	<code>geometry_msgs/msg/Twist</code>	发布速度指令
Service	<code>example_interfaces/srv/AddTwoInts</code>	两整数相加
Action	<code>turtlesim/action/RotateAbsolute</code>	海龟旋转

这些标准接口能满足通用场景，但真实机器人项目需要传递**业务专属数据**：

- 硬件状态：电机温度、就绪标志、调试信息
- 传感器融合：多源数据打包为一个消息
- 专用服务：计算矩形面积、切换工作模式

ROS 2 允许开发者**自定义接口类型**，编译后自动生成 Python 模块和 C++ 头文件，供所有节点使用。

三种接口文件



接口文件	生成产物	对应通信	定义内容
<code>.msg</code>	<code>Xxx.msg</code> → <code>Xxx</code> 类/结构体	Topic	一组字段（扁平数据结构）
<code>.srv</code>	<code>Xxx.srv</code> → <code>Xxx::Request</code> + <code>Xxx::Response</code>	Service	Request 字段 --- Response 字段
<code>.action</code>	<code>Xxx.action</code> → Goal + Result + Feedback	Action	三段式，两段 --- 分隔

本节课重点掌握 `.msg` 和 `.srv`；`.action` 的创建流程在 2.4 中已详细讲解，原理完全相同。

接口类型的命名规则

完整类型名格式：

包名/接口子目录/类型名

示例:

```
hardware_interfaces/msg/HardwareStatus
hardware_interfaces/srv/ComputeRectangleArea
count_until_interfaces/action/CountUntil
```

命名转换规则（文件 ↔ 代码）：

层面	Msg 示例	Srv 示例
定义文件	HardwareStatus.msg	ComputeRectangleArea.srv
类型全名	hardware_interfaces/msg/HardwareStatus	hardware_interfaces/srv/ComputeRectangleArea
Python 导入	from hardware_interfaces.msg import HardwareStatus	from hardware_interfaces.srv import ComputeRectangleArea
C++ 头文件	#include "hardware_interfaces/msg/hardware_status.hpp"	#include "hardware_interfaces/srv/compute_rectangle_area.hpp"
C++ 类型名	hardware_interfaces::msg::HardwareStatus	hardware_interfaces::srv::ComputeRectangleArea

规律：文件名 PascalCase → C++/Python 类型名保持 PascalCase；C++ 头文件路径和 include 名转为 snake_case。

接口包 vs 普通功能包

接口定义必须放在**纯接口包**（Interface Package）中，与普通节点包分离：

```
hardware_interfaces/      ← 纯接口包（ament_cmake，只有 .msg/.srv/.action）
├── msg/
│   └── HardwareStatus.msg
├── srv/
│   └── ComputeRectangleArea.srv
├── CMakeLists.txt
└── package.xml

py_hardware_tutorial/    ← 普通 Python 节点包（依赖 hardware_interfaces）
cpp_hardware_tutorial/   ← 普通 C++ 节点包（依赖 hardware_interfaces）
```

为什么分离？

- 接口包被多个节点包依赖，独立维护避免循环依赖
- 修改接口只需重新编译接口包，结构清晰
- 符合 ROS 2 社区标准实践（如 geometry_msgs 、 sensor_msgs 都是独立接口包）

接口的编译生成流程



`rosidl` (ROS Interface Definition Language) 是 ROS 2 的接口编译工具链，负责将文本定义转换为多语言可用的代码。

查看已有接口

```
# 列出所有已注册的接口类型
ros2 interface list

# 只看消息类型
ros2 interface list | grep msg

# 查看某个类型的完整字段定义
ros2 interface show std_msgs/msg/String
ros2 interface show turtlesim/srv/Spawn
```

关键点

- Interface 是 ROS 2 通信的"数据契约"——Topic、Service、Action 都依赖它
 - 本课程把自定义接口放在独立 `ament_cmake` 接口包中，便于跨语言复用和控制依赖；这是一项推荐结构，不是 ROS 2 强制每个项目都必须拆包
 - 类型全名格式为 `包名/msg或srv或action/类型名`
 - Python 用 `from 包名.msg import 类型名`；C++ 用 `#include "包名/msg/类型snake_case.hpp"`
 - 接口包与节点包分离是 ROS 2 标准项目结构
-

讲次 2：创建自定义 Msg 定义文件

学习目标

- 使用 `ros2 pkg create` 创建纯接口功能包 `hardware_interfaces`
- 编写 `HardwareStatus.msg` 并完成 `package.xml` 与 `CMakeLists.txt` 配置
- 成功构建接口包并用命令行验证接口已注册

知识点

创建接口功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake hardware_interfaces
```

创建消息目录和定义文件：

```
mkdir -p ~/ros2_ws/src/hardware_interfaces/msg
```

HardwareStatus.msg 定义

创建文件 `~/ros2_ws/src/hardware_interfaces/msg/HardwareStatus.msg`：

```
# 机器人硬件状态消息
# 用于周期性发布温度、电机就绪状态和调试信息

int64 temperature      # 核心温度（单位：毫摄氏度，如 45200 表示 45.2°C）
bool are_motors_ready  # 电机是否全部就绪
string debug_message   # 调试信息（可为空字符串）
```

Msg 文件语法规则：

规则	说明	示例
每行一个字段	类型 字段名	<code>int64 temperature</code>
注释	# 开头	# 核心温度
类型	基础类型或自定义类型	<code>bool</code> 、 <code>string</code> 、 <code>geometry_msgs/Pose</code>
数组	类型后加 <code>[]</code> 或 <code>[N]</code>	<code>float64[] values</code> 、 <code>int32[4] joints</code>
常量	类型 常量名=值	<code>int32 MAX_TEMP=100</code>
默认值	字段名后写默认值，不使用 <code>=</code>	<code>int32 retry_count 3</code> 、 <code>string frame_id "base_link"</code>

常用基础类型速查：

类型	含义	字节
bool	布尔值	1
int8 / uint8	8 位有/无符号整数	1
int16 / uint16	16 位有/无符号整数	2
int32 / uint32	32 位有/无符号整数	4
int64 / uint64	64 位有/无符号整数	8
float32	单精度浮点	4
float64	双精度浮点	8
string	UTF-8 字符串	变长
byte / char	原始字节/字符	1

配置 package.xml

编辑 `~/ros2_ws/src/hardware_interfaces/package.xml` :

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema" />
<package format="3">
  <name>hardware_interfaces</name>
  <version>0.0.0</version>
  <description>Custom message and service interfaces for my robot</description>
  <maintainer email="student@todo.todo">student</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <buildtool_depend>rosidl_default_generators</buildtool_depend>

  <exec_depend>rosidl_default_runtime</exec_depend>

  <member_of_group>rosidl_interface_packages</member_of_group>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

关键依赖说明:

标签	作用
rosidl_default_generators	编译期工具，读取 <code>.msg</code> 生成 Python/C++ 代码
rosidl_default_runtime	运行时所依赖的类型支持库
member_of_group: rosidl_interface_packages	向 ROS 2 声明"本包是接口包"

注意：此时只定义了 `.msg`，不需要 `action_msgs` 等额外依赖。后续添加 `.srv` 或引用其他接口类型时，再在 `DEPENDENCIES` 中声明。

配置 CMakeLists.txt

编辑 `~/ros2_ws/src/hardware_interfaces/CMakeLists.txt` :

```
cmake_minimum_required(VERSION 3.8)
project(hardware_interfaces)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/HardwareStatus.msg"
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

核心语句:

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/HardwareStatus.msg"
)
```

- `${PROJECT_NAME}` 展开为 `hardware_interfaces`
- 可一次注册多个文件: `"msg/A.msg" "msg/B.msg" "srv/C.srv"`
- 若字段引用了其他包的类型 (如 `geometry_msgs/Pose`) , 需添加 `DEPENDENCIES geometry_msgs`

构建与验证

```
cd ~/ros2_ws
colcon build --packages-select hardware_interfaces
source install/setup.bash
```

验证接口已注册:

```
ros2 interface list | grep hardware_interfaces
```

预期输出:

```
hardware_interfaces/msg/HardwareStatus
```

查看完整定义:

```
ros2 interface show hardware_interfaces/msg/HardwareStatus
```

预期输出:

```
int64 temperature
bool are_motors_ready
string debug_message
```

最终目录结构

```
hardware_interfaces/  
├─ CMakeLists.txt  
├─ package.xml  
└─ msg/  
    └─ HardwareStatus.msg
```

关键点

- 接口包使用 `ament_cmake` 构建类型，**不包含任何节点代码**
 - `package.xml` 必须包含 `rosidl_default_generators`、`rosidl_default_runtime` 和 `member_of_group: rosidl_interface_packages`
 - `CMakeLists.txt` 的核心是 `rosidl_generate_interfaces()`，列出所有 `.msg` / `.srv` / `.action` 文件路径
 - 每次修改接口定义后，必须重新 `colcon build` 接口包并 `source install/setup.bash`
 - 用 `ros2 interface show` 验证接口是否正确注册
-

讲次 3：【PY】在节点中使用自定义 Msg

学习目标

- 创建 Python 节点功能包，正确声明对 `hardware_interfaces` 的依赖
- 使用 `from hardware_interfaces.msg import HardwareStatus` 导入自定义消息
- 编写 Publisher 和 Subscriber 节点，发布与接收 `HardwareStatus` 消息

知识点

前置说明：Python 基础速览

学员有 C++ 基础但无 Python 经验，以下是本讲用到的 Python 语法对照：

Python	C++ 近似写法	说明
<code>def callback(msg):</code>	<code>void callback(const Msg & msg)</code>	函数定义
<code>self.xxx</code>	<code>this->xxx</code>	访问成员
<code>msg.temperature</code>	<code>msg.temperature</code>	访问字段（Python 用 <code>.</code> ）
<code>import rclpy</code>	<code>#include "rclcpp/rclcpp.hpp"</code>	导入模块
<code>None</code>	<code>nullptr</code>	空值
缩进（4 空格）	<code>{}</code> 花括号	代码块界定

创建 Python 节点功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python py_hardware_tutorial \
  --dependencies rclpy hardware_interfaces
```

`--dependencies hardware_interfaces` 会在 `package.xml` 中添加 ROS 依赖。`setup.py` 的 `install_requires` 用于 Python/setuptools 依赖，不要把 ROS 包名机械地再写一遍；ROS 依赖由 `rosdep` 和包清单管理。

配置 package.xml

确认 `~/ros2_ws/src/py_hardware_tutorial/package.xml` 包含：

```
<depend>rclpy</depend>
<depend>hardware_interfaces</depend>
```

配置 setup.py

编辑 `~/ros2_ws/src/py_hardware_tutorial/setup.py`，注册两个可执行入口：

```

from setuptools import find_packages, setup

package_name = 'py_hardware_tutorial'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='student',
    maintainer_email='student@todo.todo',
    description='Python tutorial for custom HardwareStatus message',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'hardware_status_publisher = py_hardware_tutorial.hardware_status_publisher:main',
            'hardware_status_subscriber = py_hardware_tutorial.hardware_status_subscriber:main',
            'rectangle_area_server = py_hardware_tutorial.rectangle_area_server:main',
        ],
    },
)

```

`rectangle_area_server` 是本讲后半部分自定义 Srv 的 Server 节点，这里先一并注册。

Publisher 节点完整代码

创建 `~/ros2_ws/src/py_hardware_tutorial/py_hardware_tutorial/hardware_status_publisher.py` :

```
import rclpy
from rclpy.node import Node
from hardware_interfaces.msg import HardwareStatus

class HardwareStatusPublisher(Node):

    def __init__(self):
        super().__init__('hardware_status_publisher')
        self.publisher_ = self.create_publisher(
            HardwareStatus, 'hardware_status', 10)
        self.timer_ = self.create_timer(1.0, self.timer_callback)
        self.temperature_ = 42000 # 初始 42.0°C (毫摄氏度)

    def timer_callback(self):
        msg = HardwareStatus()
        msg.temperature = self.temperature_
        msg.are_motors_ready = self.temperature_ < 80000
        msg.debug_message = f'Temperature: {self.temperature_ / 1000:.1f} C'

        self.publisher_.publish(msg)
        self.get_logger().info(
            f'Publishing: temp={msg.temperature}, '
            f'ready={msg.are_motors_ready}, msg="{msg.debug_message}"')

        self.temperature_ += 500 # 每次升高 0.5°C

def main(args=None):
    rclpy.init(args=args)
    node = HardwareStatusPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

代码解读：

行	说明
<code>from hardware_interfaces.msg import HardwareStatus</code>	导入自定义消息类型
<code>self.create_publisher(HardwareStatus, 'hardware_status', 10)</code>	创建 Publisher，类型为自定义 Msg
<code>msg = HardwareStatus()</code>	创建消息实例（类似 C++ 的 <code>HardwareStatus msg; </code> ）
<code>msg.temperature = ...</code>	填充字段
<code>self.publisher_.publish(msg)</code>	发布消息

Subscriber 节点完整代码

创建 `~/ros2_ws/src/py_hardware_tutorial/py_hardware_tutorial/hardware_status_subscriber.py`：

```

import rclpy
from rclpy.node import Node
from hardware_interfaces.msg import HardwareStatus

class HardwareStatusSubscriber(Node):

    def __init__(self):
        super().__init__('hardware_status_subscriber')
        self.subscription_ = self.create_subscription(
            HardwareStatus,
            'hardware_status',
            self.callback,
            10)

    def callback(self, msg):
        temp_celsius = msg.temperature / 1000.0
        self.get_logger().info(
            f'Received: {temp_celsius:.1f} C, '
            f'motors_ready={msg.are_motors_ready}, '
            f'debug="{msg.debug_message}"')

        if not msg.are_motors_ready:
            self.get_logger().warn('Motors not ready!')

def main(args=None):
    rclpy.init(args=args)
    node = HardwareStatusSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

构建与运行

```

cd ~/ros2_ws
colcon build --packages-select py_hardware_tutorial
source install/setup.bash

```

终端 1 — 运行 Publisher:

```
ros2 run py_hardware_tutorial hardware_status_publisher
```

终端 2 — 运行 Subscriber:

```
ros2 run py_hardware_tutorial hardware_status_subscriber
```

终端 3 — 命令行验证:

```
ros2 topic echo /hardware_status
```

预期输出 (节选) :

```
temperature: 42000
are_motors_ready: true
debug_message: 'Temperature: 42.0 C'
---
temperature: 42500
are_motors_ready: true
debug_message: 'Temperature: 42.5 C'
---
```

也可以用 `ros2 topic pub` 手动发布自定义消息：

```
ros2 topic pub /hardware_status hardware_interfaces/msg/HardwareStatus \
  "{temperature: 50000, are_motors_ready: false, debug_message: 'test'}" --once
```

关键点

- Python 导入自定义 Msg: `from hardware_interfaces.msg import HardwareStatus`
 - 节点包的 `package.xml` 和 `setup.py` 都必须声明对 `hardware_interfaces` 的依赖
 - 创建 Publisher/Subscriber 时，将 `HardwareStatus` 作为类型参数传入，与标准消息用法完全一致
 - 消息字段用 `msg.字段名` 访问，发布用 `publish(msg)`
 - 接口包必须先于节点包构建；修改 `.msg` 后需重新编译接口包再编译节点包
-

讲次 4：【C++】在节点中使用自定义 Msg

学习目标

- 创建 C++ 节点功能包，配置对 `hardware_interfaces` 的编译与链接依赖
- 使用 `#include "hardware_interfaces/msg/hardware_status.hpp"` 导入自定义消息
- 编写 Publisher 和 Subscriber 节点，发布与接收 `HardwareStatus` 消息

知识点

创建 C++ 节点功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake cpp_hardware_tutorial \
  --dependencies rclcpp hardware_interfaces
```

配置 package.xml

确认 `~/ros2_ws/src/cpp_hardware_tutorial/package.xml` 包含：

```
<depend>rclcpp</depend>
<depend>hardware_interfaces</depend>
```

配置 CMakeLists.txt

编辑 `~/ros2_ws/src/cpp_hardware_tutorial/CMakeLists.txt`：

```

cmake_minimum_required(VERSION 3.8)
project(cpp_hardware_tutorial)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(hardware_interfaces REQUIRED)

# — Publisher 可执行文件 —
add_executable(hardware_status_publisher src/hardware_status_publisher.cpp)
ament_target_dependencies(hardware_status_publisher rclcpp hardware_interfaces)

# — Subscriber 可执行文件 —
add_executable(hardware_status_subscriber src/hardware_status_subscriber.cpp)
ament_target_dependencies(hardware_status_subscriber rclcpp hardware_interfaces)

# — 自定义 Srv 的 Server（本讲后半部分） —
add_executable(rectangle_area_server src/rectangle_area_server.cpp)
ament_target_dependencies(rectangle_area_server rclcpp hardware_interfaces)

install(TARGETS
  hardware_status_publisher
  hardware_status_subscriber
  rectangle_area_server
  DESTINATION lib/${PROJECT_NAME})

ament_package()

```

对已经由独立包导出的接口，`find_package(hardware_interfaces REQUIRED)` 加 `ament_target_dependencies(... hardware_interfaces)` 会带入头文件与类型支持链接依赖。`rosidl_get_typesupport_target()` 主要用于同一个 CMake 包既生成接口又直接链接这些接口的特殊结构，本例不需要手工调用。

Publisher 节点完整代码

创建 `~/ros2_ws/src/cpp_hardware_tutorial/src/hardware_status_publisher.cpp`：

```

#include <iomanip>
#include <sstream>

#include "rclcpp/rclcpp.hpp"
#include "hardware_interfaces/msg/hardware_status.hpp"

class HardwareStatusPublisher : public rclcpp::Node
{
public:
    HardwareStatusPublisher()
    : Node("hardware_status_publisher"), temperature_(42000)
    {
        publisher_ = this->create_publisher<hardware_interfaces::msg::HardwareStatus>(
            "hardware_status", 10);
        timer_ = this->create_wall_timer(
            std::chrono::seconds(1),
            std::bind(&HardwareStatusPublisher::timer_callback, this));
    }

private:
    void timer_callback()
    {
        auto msg = hardware_interfaces::msg::HardwareStatus();
        msg.temperature = temperature_;
        msg.are_motors_ready = temperature_ < 80000;
        std::ostringstream debug_stream;
        debug_stream << "Temperature: " << std::fixed << std::setprecision(1)
            << temperature_ / 1000.0 << " C";
        msg.debug_message = debug_stream.str();

        publisher_->publish(msg);
        RCLCPP_INFO(this->get_logger(),
            "Publishing: temp=%ld, ready=%s, msg=\"%s\"",
            msg.temperature,
            msg.are_motors_ready ? "true" : "false",
            msg.debug_message.c_str());

        temperature_ += 500;
    }

    rclcpp::Publisher<hardware_interfaces::msg::HardwareStatus>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
    int64_t temperature_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<HardwareStatusPublisher>());
    rclcpp::shutdown();
    return 0;
}

```

Subscriber 节点完整代码

创建 `~/ros2_ws/src/cpp_hardware_tutorial/src/hardware_status_subscriber.cpp` :


```

#include "rclcpp/rclcpp.hpp"
#include "hardware_interfaces/msg/hardware_status.hpp"

class HardwareStatusSubscriber : public rclcpp::Node
{
public:
    HardwareStatusSubscriber()
    : Node("hardware_status_subscriber")
    {
        subscription_ = this->create_subscription<hardware_interfaces::msg::HardwareStatus>(
            "hardware_status", 10,
            std::bind(&HardwareStatusSubscriber::callback, this, std::placeholders::_1));
    }

private:
    void callback(const hardware_interfaces::msg::HardwareStatus::SharedPtr msg)
    {
        double temp_celsius = msg->temperature / 1000.0;
        RCLCPP_INFO(this->get_logger(),
            "Received: %.1f C, motors_ready=%s, debug=\"%s\"",
            temp_celsius,
            msg->are_motors_ready ? "true" : "false",
            msg->debug_message.c_str());

        if (!msg->are_motors_ready) {
            RCLCPP_WARN(this->get_logger(), "Motors not ready!");
        }
    }

    rclcpp::Subscription<hardware_interfaces::msg::HardwareStatus>::SharedPtr subscription_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<HardwareStatusSubscriber>());
    rclcpp::shutdown();
    return 0;
}

```

Python vs C++ 自定义 Msg 用法对照

操作	Python	C++
导入	<code>from hardware_interfaces.msg import HardwareStatus</code>	<code>#include "hardware_interfaces/msg/hardware_status.hpp"</code>
类型名	<code>HardwareStatus</code>	<code>hardware_interfaces::msg::HardwareStatus</code>
创建实例	<code>msg = HardwareStatus()</code>	<code>auto msg = HardwareStatus();</code>
访问字段	<code>msg.temperature</code>	<code>msg.temperature / msg->temperature</code>
创建 Publisher	<code>create_publisher(HardwareStatus, ...)</code>	<code>create_publisher<HardwareStatus>(...)</code>
发布	<code>publisher_.publish(msg)</code>	<code>publisher_->publish(msg)</code>

构建与运行

```
cd ~/ros2_ws
colcon build --packages-select cpp_hardware_tutorial
source install/setup.bash
```

终端 1:

```
ros2 run cpp_hardware_tutorial hardware_status_publisher
```

终端 2:

```
ros2 run cpp_hardware_tutorial hardware_status_subscriber
```

关键点

- C++ 头文件路径: `#include "hardware_interfaces/msg/hardware_status.hpp"` (snake_case)
 - C++ 类型全名: `hardware_interfaces::msg::HardwareStatus`
 - 外部接口包通过 `ament_target_dependencies(target rclcpp hardware_interfaces)` 传递编译与链接依赖; 不要重复手工链接类型支持目标
 - 回调中消息以 `SharedPtr` 传递, 字段访问用 `msg->字段名`
 - 自定义 Msg 在 C++ 中的用法与 `std_msgs` 等标准消息完全一致, 只是 include 和类型名不同
-

讲次 5：创建自定义 Srv

学习目标

- 理解 `.srv` 文件的 Request/Response 分隔格式
- 在 `hardware_interfaces` 包中添加 `ComputeRectangleArea.srv`
- 更新 `CMakeLists.txt` 重新构建，并用命令行验证

知识点

Srv 文件格式

Service 接口使用 `.srv` 文件定义，用 `---` 分隔 Request 和 Response：

```
# Request 部分（--- 上方）
字段类型 字段名
---
# Response 部分（--- 下方）
字段类型 字段名
```

与 Msg 的对比：

特征	<code>.msg</code>	<code>.srv</code>
分隔符	无	<code>---</code> 分隔 Request / Response
生成产物	一个消息结构体	<code>Request</code> + <code>Response</code> 两个结构体
使用场景	Topic 发布/订阅	Service 请求/响应

ComputeRectangleArea.srv 定义

创建目录和文件：

```
mkdir -p ~/ros2_ws/src/hardware_interfaces/srv
```

创建 `~/ros2_ws/src/hardware_interfaces/srv/ComputeRectangleArea.srv`：

```
# 请求：矩形的长和宽
float64 length
float64 width
---
# 响应：计算得到的面积
float64 area
```

更新 CMakeLists.txt

编辑 `~/ros2_ws/src/hardware_interfaces/CMakeLists.txt`，在 `rosidl_generate_interfaces` 中添加 `srv` 文件：

```

cmake_minimum_required(VERSION 3.8)
project(hardware_interfaces)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/HardwareStatus.msg"
  "srv/ComputeRectangleArea.srv"
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()

```

package.xml 无需额外修改—— rosidl_default_generators 和 rosidl_default_runtime 同时支持 .msg 和 .srv 。

构建与验证

```

cd ~/ros2_ws
colcon build --packages-select hardware_interfaces
source install/setup.bash

```

验证：

```
ros2 interface list | grep hardware_interfaces
```

预期输出：

```

hardware_interfaces/msg/HardwareStatus
hardware_interfaces/srv/ComputeRectangleArea

```

查看 Srv 定义：

```
ros2 interface show hardware_interfaces/srv/ComputeRectangleArea
```

预期输出：

```

float64 length
float64 width
---
float64 area

```

在节点中使用自定义 Srv（导入方式）

接口包构建完成后，其他节点包即可像使用标准 Srv 一样使用它。导入方式：

Python：

```

from hardware_interfaces.srv import ComputeRectangleArea

# 创建 Server
self.srv_ = self.create_service(
    ComputeRectangleArea, 'compute_rectangle_area', self.callback)

# 回调函数
def callback(self, request, response):
    response.area = request.length * request.width
    return response

```

C++:

```

#include "hardware_interfaces/srv/compute_rectangle_area.hpp"

// 创建 Server
srv_ = this->create_service<hardware_interfaces::srv::ComputeRectangleArea>(
    "compute_rectangle_area",
    std::bind(&MyNode::callback, this, std::placeholders::_1, std::placeholders::_2));

// 回调函数
void callback(
    const std::shared_ptr<hardware_interfaces::srv::ComputeRectangleArea::Request> request,
    std::shared_ptr<hardware_interfaces::srv::ComputeRectangleArea::Response> response)
{
    response->area = request->length * request->width;
}

```

命令行手动调用（先启动上面的 Server 节点；仓库参考实现已注册为可执行文件，可直接运行）：

```

# 终端 1: 启动 Server (Python 或 C++ 二选一)
ros2 run py_hardware_tutorial rectangle_area_server
# 或 ros2 run cpp_hardware_tutorial rectangle_area_server

# 终端 2: 手动调用
ros2 service call /compute_rectangle_area \
    hardware_interfaces/srv/ComputeRectangleArea \
    "{length: 5.0, width: 3.0}"

```

预期响应：

```

response:
hardware_interfaces.srv.ComputeRectangleArea_Response(area=15.0)

```

完整的 Service Server/Client 编写在 2.3 中已详细讲解。本讲重点在于**定义接口**；使用方式与 example_interfaces/srv/AddTwoInts 完全相同，只是换成自己的包名和类型名。

最终接口包目录结构

```

hardware_interfaces/
├── CMakeLists.txt
├── package.xml
├── msg/
│   └── HardwareStatus.msg
├── srv/
│   └── ComputeRectangleArea.srv

```

关键点

- `.srv` 文件用 `---` 分隔 Request (上方) 和 Response (下方)
 - 同一个接口包可同时包含 `.msg`、`.srv`、`.action`，在 `rosidl_generate_interfaces()` 中一并列出
 - Python 导入: `from hardware_interfaces.srv import ComputeRectangleArea`
 - C++ 头文件: `#include "hardware_interfaces/srv/compute_rectangle_area.hpp"`
 - 添加新接口后必须重新编译接口包，依赖它的所有节点包也需要重新编译
-

讲次 6：接口命令行查看与调试

学习目标

- 熟练使用 `ros2 interface list` 浏览系统中所有已注册接口
- 使用 `ros2 interface show` 查看任意接口类型的完整字段定义
- 使用 `ros2 interface package` 查看指定包导出的所有接口

知识点

ros2 interface list — 列出所有接口

```
ros2 interface list
```

输出系统中所有已注册的 `.msg`、`.srv`、`.action` 类型，按字母排序。在 Jazzy 上通常有数百个（含标准接口和自定义接口）。

过滤技巧：

```
# 只看自定义接口
ros2 interface list | grep hardware_interfaces

# 只看消息类型
ros2 interface list | grep msg

# 只看服务类型
ros2 interface list | grep srv

# 只看 Action 类型
ros2 interface list | grep action

# 查看某个标准包的所有接口
ros2 interface list | grep geometry_msgs

# 查看传感器相关接口
ros2 interface list | grep sensor_msgs
```

ros2 interface show — 查看接口定义

```
ros2 interface show <类型全名>
```

查看自定义 Msg：

```
ros2 interface show hardware_interfaces/msg/HardwareStatus
```

输出：

```
int64 temperature
bool are_motors_ready
string debug_message
```

查看自定义 Srv：

```
ros2 interface show hardware_interfaces/srv/ComputeRectangleArea
```

输出:

```
float64 length
float64 width
---
float64 area
```

查看标准接口 (对比学习) :

```
ros2 interface show std_msgs/msg/String
ros2 interface show geometry_msgs/msg/Twist
ros2 interface show example_interfaces/srv/AddTwoInts
ros2 interface show turtlesim/action/RotateAbsolute
```

`ros2 interface show` 的输出格式与 `.msg` / `.srv` / `.action` 源文件一致, 是编写 `ros2 topic pub`、`ros2 service call` 命令的数据格式参考。

ros2 interface package — 查看包导出的接口

```
ros2 interface package <包名>
```

查看自定义接口包:

```
ros2 interface package hardware_interfaces
```

输出:

```
hardware_interfaces/msg/HardwareStatus
hardware_interfaces/srv/ComputeRectangleArea
```

查看标准接口包:

```
ros2 interface package geometry_msgs
ros2 interface package sensor_msgs
ros2 interface package example_interfaces
```

接口调试 workflow

开发自定义接口时, 推荐以下调试流程:


```
1. 编写 .msg / .srv 文件
  |
  v
2. colcon build --packages-select hardware_interfaces
  |
  v
3. source install/setup.bash
  |
  v
4. ros2 interface package hardware_interfaces    ← 确认包导出正确
  |
  v
5. ros2 interface show hardware_interfaces/msg/HardwareStatus ← 确认字段正确
  |
  v
6. 编写节点代码，编译节点包
  |
  v
7. ros2 topic echo / ros2 service call ← 运行时验证
```

常见问题排查

现象	原因	解决方法
<code>ros2 interface list</code> 中找不到自定义类型	未编译或未 source	<code>colcon build + source install/setup.bash</code>
<code>ros2 interface show</code> 报 <code>Unknown package</code>	包名拼写错误或未安装	检查包名，确认 <code>ros2 pkg list grep hardware</code>
节点编译报 <code>No such file or directory</code> (头文件)	接口包未先编译	先 <code>colcon build --packages-select hardware_interfaces</code>
C++ 链接 <code>undefined reference</code>	接口包未正确导出，或消费包缺少依赖	确认接口包 <code>ament_export_dependencies(rosidl_default_runtime)</code> ，消费目标使用 <code>ament_target_dependencies(... hardware_interfaces)</code>
Python 报 <code>ModuleNotFoundError</code>	未 source 或依赖未声明	<code>source install/setup.bash</code> ，检查 <code>package.xml</code>
修改 <code>.msg</code> 后节点行为异常	节点包未重新编译	先编译接口包，再编译所有依赖它的节点包
<code>ros2 topic pub</code> 字段名报错	字段名或类型不匹配	用 <code>ros2 interface show</code> 确认正确字段名和类型

结合其他 CLI 工具

接口信息与 Topic/Service 命令配合使用（第 1、4、6 步需要对应节点在运行：先 `ros2 run py_hardware_tutorial hardware_status_publisher`、`ros2 run py_hardware_tutorial rectangle_area_server`）：

```
# 1. 查看 Topic 类型
ros2 topic info /hardware_status -v
# 输出 Type: hardware_interfaces/msg/HardwareStatus

# 2. 查看该类型的字段定义
ros2 interface show hardware_interfaces/msg/HardwareStatus

# 3. 按正确格式发布消息
ros2 topic pub /hardware_status hardware_interfaces/msg/HardwareStatus \
    "{temperature: 45000, are_motors_ready: true, debug_message: 'OK'}" --once

# 4. 查看 Service 类型
ros2 service type /compute_rectangle_area

# 5. 查看 Srv 定义
ros2 interface show hardware_interfaces/srv/ComputeRectangleArea

# 6. 按正确格式调用服务
ros2 service call /compute_rectangle_area \
    hardware_interfaces/srv/ComputeRectangleArea "{length: 4.0, width: 2.5}"
```

关键点

- `ros2 interface list` 列出所有已注册接口，配合 `grep` 快速过滤
 - `ros2 interface show <类型全名>` 查看完整字段定义，是编写 `pub/call` 命令的格式参考
 - `ros2 interface package <包名>` 查看某个包导出的所有接口
 - 接口调试的标准流程：编写 → 编译 → `source` → `interface package` → `interface show` → 编写节点 → 运行时验证
 - 修改接口定义后，必须按依赖顺序重新编译：接口包 → 节点包
-

本节总结

命令速查表

命令	说明
<code>ros2 pkg create --build-type ament_cmake hardware_interfaces</code>	创建纯接口功能包
<code>colcon build --packages-select hardware_interfaces</code>	编译接口包
<code>ros2 interface list</code>	列出所有已注册的接口类型
<code>ros2 interface list \ grep hardware_interfaces</code>	过滤查看自定义接口
<code>ros2 interface list \ grep msg</code>	只列出消息类型
<code>ros2 interface list \ grep srv</code>	只列出服务类型
<code>ros2 interface show hardware_interfaces/msg/HardwareStatus</code>	查看自定义 Msg 字段定义
<code>ros2 interface show hardware_interfaces/srv/ComputeRectangleArea</code>	查看自定义 Srv 字段定义
<code>ros2 interface show <type></code>	查看任意接口类型的完整定义
<code>ros2 interface package hardware_interfaces</code>	查看指定包导出的所有接口
<code>ros2 interface package geometry_msgs</code>	查看标准接口包的所有接口
<code>ros2 run py_hardware_tutorial hardware_status_publisher</code>	运行 Python 自定义 Msg Publisher
<code>ros2 run py_hardware_tutorial hardware_status_subscriber</code>	运行 Python 自定义 Msg Subscriber
<code>ros2 run cpp_hardware_tutorial hardware_status_publisher</code>	运行 C++ 自定义 Msg Publisher
<code>ros2 run cpp_hardware_tutorial hardware_status_subscriber</code>	运行 C++ 自定义 Msg Subscriber
<code>ros2 topic echo /hardware_status</code>	查看 HardwareStatus 消息内容
<code>ros2 topic pub /hardware_status hardware_interfaces/msg/HardwareStatus "{...}"</code>	手动发布自定义消息
<code>ros2 service call /compute_rectangle_area hardware_interfaces/srv/ComputeRectangleArea "{length: 5.0, width: 3.0}"</code>	手动调用自定义服务

核心配置速查

配置项	位置	内容
接口包依赖	hardware_interfaces/package.xml	<code>rosidl_default_generators + rosidl_default_runtime + member_of_group: rosidl_interface_packages</code>
接口生成	hardware_interfaces/CMakeLists.txt	<code>rosidl_generate_interfaces(\${PROJECT_NAME} "msg/..." "srv/...")</code>
Python 导入 Msg	节点 .py 文件	<code>from hardware_interfaces.msg import HardwareStatus</code>
Python 导入 Srv	节点 .py 文件	<code>from hardware_interfaces.srv import ComputeRectangleArea</code>
C++ 导入 Msg	节点 .cpp 文件	<code>#include "hardware_interfaces/msg/hardware_status.hpp"</code>
C++ 导入 Srv	节点 .cpp 文件	<code>#include "hardware_interfaces/srv/compute_rectangle_area.hpp"</code>
C++ 链接	节点 CMakeLists.txt	<code>ament_target_dependencies(target rclcpp hardware_interfaces)</code>
节点包依赖	节点 package.xml	<code><depend>hardware_interfaces</depend></code>

三种接口文件对比

维度	.msg	.srv	.action
用途	Topic 消息	Service 请求/响应	Action 目标/反馈/结果
分隔符	无	一个 ---	两个 ---
生成产物	一个结构体	Request + Response	Goal + Result + Feedback
Python 导入	<code>from pkg.msg import Xxx</code>	<code>from pkg.srv import Xxx</code>	<code>from pkg.action import Xxx</code>
C++ 头文件	<code>pkg/msg/xxx.hpp</code>	<code>pkg/srv/xxx.hpp</code>	<code>pkg/action/xxx.hpp</code>
本课程示例	HardwareStatus.msg	ComputeRectangleArea.srv	CountUntil.action (2.4)

下一步

下一节 **2.6 Launch 文件** 中，我们将学习如何用 Launch 文件一次性启动多个节点——告别开多个终端手动 `ros2 run` 的低效方式。你将掌握 Python Launch 文件的编写、`ros2 launch` 命令的使用、节点参数与重映射配置，以及如何用 Launch 文件同时启动本节的 Publisher 和 Subscriber，实现一键启动整个硬件状态监控演示。